ITU Source Code for Copyright Application

Generated on: March 24, 2026

PART 1: FIRST 25 PAGES (or equivalent)

```
// FILE: backend/config/cloudinary.js
const cloudinary = require('cloudinary').v2;
require('dotenv').config();

// Configure Cloudinary
cloudinary.config({
  cloud_name: process.env.CLOUDINARY_CLOUD_NAME,
  api_key: process.env.CLOUDINARY_API_KEY,
  api_secret: process.env.CLOUDINARY_API_SECRET
});

// Helper function to upload image to Cloudinary
const uploadToCloudinary = async (filePath, folder = 'itu') => {
  try {
    const result = await cloudinary.uploader.upload(filePath, {
      folder: folder,
      resource_type: 'auto',
      transformation: [
        { quality: 'auto' },
        { fetch_format: 'auto' }
      ]
    });
    return {
      url: result.secure_url,
      public_id: result.public_id,
      width: result.width,
      height: result.height,
      format: result.format
    };
  } catch (error) {
    console.error('Cloudinary upload error:', error);
    throw error;
  }
};

// Helper function to upload from buffer (for multer)
const uploadBufferToCloudinary = async (buffer, folder = 'itu', filename = null) => {
  return new Promise((resolve, reject) => {
    const uploadStream = cloudinary.uploader.upload_stream(
      {
        folder: folder,
        resource_type: 'auto',
        transformation: [
          { quality: 'auto' },
          { fetch_format: 'auto' }
        ],
        public_id: filename ? `${folder}/${filename}` : undefined
      },
      (error, result) => {
        if (error) {
          console.error('Cloudinary upload error:', error);
          reject(error);
        } else {
          resolve({
            url: result.secure_url,
            public_id: result.public_id,
            width: result.width,
            height: result.height,
            format: result.format
          });
        }
```

```
        }
      );
      uploadStream.end(buffer);
    });
  };

  // Helper function to delete image from Cloudinary
  const deleteFromCloudinary = async (publicId) => {
    try {
      const result = await cloudinary.uploader.destroy(publicId);
      return result;
    } catch (error) {
      console.error('Cloudinary delete error:', error);
      throw error;
    }
  };

  module.exports = {
    cloudinary,
    uploadToCloudinary,
    uploadBufferToCloudinary,
    deleteFromCloudinary
  };



  // FILE: backend/config/db.js
  const mongoose = require('mongoose');
  require('dotenv').config();

  const connectDB = async () => {
    try {
      // Check if MONGO_URI is set
      if (!process.env.MONGO_URI && !process.env.MONGODB_URI) {
        console.error('❌ MongoDB URI not found in environment variables');
        console.error('Please set MONGO_URI or MONGODB_URI in your environment variables');
        process.exit(1);
      }

      const mongoUri = process.env.MONGO_URI || process.env.MONGODB_URI;

      const conn = await mongoose.connect(mongoUri, {
        useNewUrlParser: true,
        useUnifiedTopology: true,
        serverSelectionTimeoutMS: 5000, // Timeout after 5s instead of 30s
        socketTimeoutMS: 45000, // Close sockets after 45s of inactivity
      });

      console.log(`✅ MongoDB Connected: ${conn.connection.host}`);
      console.log(`✅ Database: ${conn.connection.name}`);
    } catch (error) {
      console.error(`❌ MongoDB connection failed: ${error.message}`);
      console.error('Please check:');
      console.error('1. MONGO_URI or MONGODB_URI is set correctly');
      console.error('2. MongoDB server is accessible');
      console.error('3. Network/firewall settings allow connection');
      process.exit(1); // Exit the app if DB connection fails
    }
  };

  module.exports = connectDB;


  // FILE: backend/config/email.js
  const { Resend } = require('resend');
  require('dotenv').config();

  let resendClient = null;

  // Initialize Resend client
  const getResendClient = () => {
    const apiKey = process.env.RESEND_API_KEY;

    if (!apiKey) {
```

```
    console.error('❌ RESEND_API_KEY is not set in environment variables');
    console.error('Please set RESEND_API_KEY in your environment variables');
    return null;
  }

  // Always create a new client to ensure we have the latest API key
  // (in case env vars changed after startup)
  try {
    resendClient = new Resend(apiKey);
    console.log('✅ Resend client initialized/refreshed');
    return resendClient;
  } catch (error) {
    console.error('❌ Error creating Resend client:', error.message);
    return null;
    }
  };

// Log email configuration status on startup
console.log('\n📧 ========== EMAIL CONFIGURATION STATUS (RESEND) ==========');
console.log('  RESEND_API_KEY:', process.env.RESEND_API_KEY ? '***configured***' : '❌ NOT SET');
console.log('  RESEND_FROM_EMAIL:', process.env.RESEND_FROM_EMAIL || 'not set (will use default)');
console.log('  Resend Client:', getResendClient() ? '✅ Initialized' : '❌ Failed');
console.log('==========================================================\n');

// Get email from address
const getEmailFrom = () => {
  // Resend format: "Name <email@domain.com>" or just "email@domain.com"
  let fromEmail = process.env.RESEND_FROM_EMAIL || process.env.EMAIL_USER;
  const fromName = process.env.RESEND_FROM_NAME || 'Indian Taekwondo Union';

  // Resend doesn't allow Gmail addresses without domain verification
  // Use Resend's default domain for testing, or verify your own domain
  if (!fromEmail || fromEmail.includes('@gmail.com') || fromEmail.includes('@yahoo.com') ||
fromEmail.includes('@hotmail.com')) {
    // Use Resend's default domain (no verification needed)
    fromEmail = 'onboarding@resend.dev';
    console.warn('⚠️  Using Resend default domain (onboarding@resend.dev) because Gmail/Yahoo/Hotmail
domains require verification.');
    console.warn('   To use your own email, verify your domain at: https://resend.com/domains');
  }

  // If email contains @, use it; otherwise format it properly
  if (fromEmail.includes('@')) {
    return `${fromName} <${fromEmail}>`;
  }
  return fromEmail;
};

// Helper function to send email using Resend
const sendEmail = async (mailOptions) => {
  console.log('\n\n🚀🚀🚀 SEND EMAIL FUNCTION CALLED (RESEND) 🚀🚀🚀');
  console.log('Timestamp:', new Date().toISOString());

  try {
    console.log('\n========== EMAIL SENDING ATTEMPT ==========');
    console.log('To:', mailOptions.to);
    console.log('Subject:', mailOptions.subject);
    console.log('From:', mailOptions.from || getEmailFrom());

    const client = getResendClient();

    if (!client) {
      const errorMsg = 'Resend API key is not configured. Please set RESEND_API_KEY environment
variable.';
      console.error('\n========== EMAIL ERROR DETAILS ==========');
      console.error('ERROR TYPE: Resend Not Configured');
      console.error('ERROR MESSAGE:', errorMsg);
      console.error('RESEND_API_KEY:', process.env.RESEND_API_KEY ? '***set***' : '❌ NOT SET');
      console.error('==========================================\n');
      throw new Error(errorMsg);
    }
```

```
    // Parse from email - Resend accepts "email@domain.com" or "Name <email@domain.com>"
    let fromEmail = mailOptions.from || getEmailFrom();

    // Check if fromEmail contains Gmail/Yahoo/Hotmail (not allowed without domain verification)
    // Extract email from "Name <email>" format if needed
    let emailOnly = fromEmail.replace(/.*<|>.*/g, '').trim() || fromEmail;
    if (emailOnly.includes('@gmail.com') || emailOnly.includes('@yahoo.com') ||
emailOnly.includes('@hotmail.com')) {
      // Use Resend's default domain instead
      emailOnly = 'onboarding@resend.dev';
      console.warn('⚠️  Gmail/Yahoo/Hotmail addresses require domain verification. Using Resend default
domain.');
    }

    // Format properly - Resend requires "Name <email@domain.com>" format
    const fromName = process.env.RESEND_FROM_NAME || 'Indian Taekwondo Union';

    // Always format as "Name <email@domain.com>"
    fromEmail = `${fromName} <${emailOnly}>`;

    // Validate format
    const emailFormatRegex = /^.+?\s*<[^\s<>]+@[^\s<>]+>$/;
    if (!emailFormatRegex.test(fromEmail)) {
      console.error('❌ Invalid email format:', fromEmail);
      throw new Error(`Invalid from email format: ${fromEmail}. Must be "Name <email@domain.com>"`);
    }

    // Ensure 'to' is an array
    const toEmails = Array.isArray(mailOptions.to) ? mailOptions.to : [mailOptions.to];

    // Send email using Resend
    console.log('Sending email via Resend API...');
    console.log('From (before send):', JSON.stringify(fromEmail));
    console.log('To:', toEmails);

    // Ensure fromEmail is a clean string without extra characters
    const cleanFromEmail = fromEmail.trim();

    const { data, error } = await client.emails.send({
      from: cleanFromEmail,
      to: toEmails,
      subject: mailOptions.subject,
      html: mailOptions.html,
      text: mailOptions.text || (mailOptions.html ? mailOptions.html.replace(/<[^>]*>/g, '') :
undefined)
    });

    if (error) {
      console.error('\n========== RESEND API ERROR ==========');
      console.error('ERROR:', JSON.stringify(error, null, 2));
      console.error('====================================\n');
      throw new Error(error.message || 'Resend API error');
    }

    console.log('\n========== ✅ EMAIL SENT SUCCESSFULLY ==========');
    console.log('Message ID:', data?.id);
    console.log('To:', mailOptions.to);
    console.log('Subject:', mailOptions.subject);
    console.log('Resend Response:', JSON.stringify(data, null, 2));
    console.log('============================================\n');

    return {
      success: true,
      info: {
        messageId: data?.id,
        response: 'Email sent via Resend',
        accepted: [mailOptions.to],
        rejected: []
      }
    };
  } catch (error) {
    console.error('\n========== ❌ EMAIL ERROR DETAILS ==========');
    console.error('ERROR TYPE:', error.name || 'Unknown');
    console.error('ERROR MESSAGE:', error.message);
```

```
    console.error('ERROR STACK:', error.stack || 'N/A');
    console.error('\n--- Email Details ---');
    console.error('To:', mailOptions.to);
    console.error('Subject:', mailOptions.subject);
    console.error('From:', mailOptions.from || getEmailFrom());
    console.error('\n--- Configuration Check ---');
    console.error('RESEND_API_KEY:', process.env.RESEND_API_KEY ? '***configured***' : '❌ NOT SET');
    console.error('RESEND_FROM_EMAIL:', process.env.RESEND_FROM_EMAIL || 'not set');
    console.error('==========================================\n');

    console.error('FULL ERROR OBJECT:', JSON.stringify(error, Object.getOwnPropertyNames(error), 2));

    return { success: false, error: error.message, fullError: error };
  }
};

// Legacy transporter getter for backward compatibility (returns null, use sendEmail instead)
const getTransporter = () => {
  console.warn('⚠️  getTransporter() called - Resend doesn\'t use transporters. Use sendEmail()
instead.');
  return null;
};

// Legacy createTransporter for backward compatibility
const createTransporter = () => {
  console.warn('⚠️  createTransporter() called - Resend doesn\'t use transporters. Use sendEmail()
instead.');
  return null;
};

module.exports = {
  get transporter() {
    return getTransporter();
  },
  getEmailFrom,
  createTransporter,
  sendEmail,
  getResendClient
};


// FILE: backend/config/firebase.js
const admin = require('firebase-admin');
const path = require('path');
const fs = require('fs');
require('dotenv').config();

// Initialize Firebase Admin SDK
let firebaseApp = null;

const initializeFirebase = () => {
  try {
    // Check if Firebase is already initialized
    if (admin.apps.length > 0) {
      firebaseApp = admin.app();
      console.log('✅ Firebase already initialized');
      return firebaseApp;
    }

    // Option 1: Using service account JSON (Recommended for production)
    if (process.env.FIREBASE_SERVICE_ACCOUNT) {
      const serviceAccount = JSON.parse(process.env.FIREBASE_SERVICE_ACCOUNT);
      firebaseApp = admin.initializeApp({
        credential: admin.credential.cert(serviceAccount),
        databaseURL: process.env.FIREBASE_DATABASE_URL
      });
      console.log('✅ Firebase initialized with service account');
      return firebaseApp;
    }

    // Option 2: Using service account file path
    if (process.env.FIREBASE_SERVICE_ACCOUNT_PATH) {
      try {
        // Resolve the path relative to the backend directory
```

```
        const serviceAccountPath = path.resolve(__dirname, '..',
process.env.FIREBASE_SERVICE_ACCOUNT_PATH.replace('./', ''));

        // Check if file exists before requiring it
        if (!fs.existsSync(serviceAccountPath)) {
          console.warn(`⚠️  Firebase service account file not found at: ${serviceAccountPath}`);
          console.warn('⚠️  Firebase features will be disabled. Using static data fallback.');
          return null;
        }

        const serviceAccount = require(serviceAccountPath);
        firebaseApp = admin.initializeApp({
          credential: admin.credential.cert(serviceAccount),
          databaseURL: process.env.FIREBASE_DATABASE_URL
        });
        console.log('✅ Firebase initialized with service account file');
        return firebaseApp;
      } catch (fileError) {
        console.warn(`⚠️  Error loading Firebase service account file: ${fileError.message}`);
        console.warn('⚠️  Firebase features will be disabled. Using static data fallback.');
        return null;
      }
    }

    // Option 3: Using individual credentials from .env
    if (process.env.FIREBASE_PROJECT_ID && process.env.FIREBASE_PRIVATE_KEY &&
process.env.FIREBASE_CLIENT_EMAIL) {
      firebaseApp = admin.initializeApp({
        credential: admin.credential.cert({
          projectId: process.env.FIREBASE_PROJECT_ID,
          privateKey: process.env.FIREBASE_PRIVATE_KEY.replace(/\\n/g, '\n'),
          clientEmail: process.env.FIREBASE_CLIENT_EMAIL,
        }),
        databaseURL: process.env.FIREBASE_DATABASE_URL
      });
      console.log('✅ Firebase initialized with environment variables');
      return firebaseApp;
    }

    console.warn('⚠️  Firebase credentials not found. Firebase features will be disabled.');
    console.warn('⚠️  The application will use static data fallback for states and districts.');
    return null;
  } catch (error) {
    console.warn('⚠️  Firebase initialization error:', error.message);
    console.warn('⚠️  Firebase features will be disabled. Using static data fallback.');
    return null;
  }
};

// Get Firebase Admin instance
const getFirebaseAdmin = () => {
  if (!firebaseApp) {
    firebaseApp = initializeFirebase();
  }
  return firebaseApp;
};

// Get Firestore database
const getFirestore = () => {
  const app = getFirebaseAdmin();
  if (!app) return null;

  // Get Firestore instance - try with database ID if specified
  const dbId = process.env.FIREBASE_DATABASE_ID || '(default)';
  try {
    return admin.firestore(app);
  } catch (error) {
    console.error('Error getting Firestore:', error);
    return null;
  }
};
```

```
// Get Realtime Database
const getDatabase = () => {
  const app = getFirebaseAdmin();
  if (!app) return null;
  return admin.database();
};

module.exports = {
  initializeFirebase,
  getFirebaseAdmin,
  getFirestore,
  getDatabase,
  admin
};



// FILE: backend/controllers/adminController.js
const adminService = require('../services/adminService');
const News = require('../models/News')
const Contact = require('../models/Contact')
const Slider = require('../models/Slider');
const Gallery = require('../models/Gallery')
const SelfDefenceSlider = require('../models/SelfDefenceSlider');
const CategorySlider = require('../models/CategorySlider');
const Player = require('../models/Players');
const path = require("path")
const multer = require('multer');
const fs = require('fs');
const emailConfig = require('../config/email');
const { sendEmail } = require('../config/email');
const getEmailFrom = emailConfig.getEmailFrom;
const AccelerationForm = require('../models/AccelerationForm');
const { uploadBufferToCloudinary, deleteFromCloudinary } = require('../config/cloudinary');

const { generateTokens } = require('../utils/jwt');

exports.login = async (req, res) => {
  try {
    const admin = await adminService.loginAdmin(req.body);

    // Generate JWT tokens
    const tokens = generateTokens({
      userId: admin._id.toString(),
      email: admin.email,
      role: 'admin'
    });

    // Set session (for backward compatibility)
    req.session.admin = admin._id;

    res.status(200).json({
      success: true,
      message: 'Login successful',
      data: {
        user: admin,
        ...tokens
      }
    });
  } catch (err) {
    res.status(401).json({
      success: false,
      error: err.message
    });
  }
};

exports.logout = (req, res) => {
  // Destroy session
  req.session.destroy((err) => {
    if (err) {
      return res.status(500).json({
        success: false,
        error: 'Failed to logout'
      });
```

```
    }

    res.status(200).json({
      success: true,
      message: 'Logged out successfully'
    });
  });
};



exports.getContacts = async (req, res) => {
  try {
    const contacts = await Contact.find().sort({ createdAt: -1 });
    res.status(200).json(contacts);
  } catch (error) {
    res.status(500).json({ error: "Failed to fetch contacts" });
  }
};

// Delete contact
exports.deleteContact = async (req, res) => {
  try {
    const { id } = req.params;
    const contact = await Contact.findById(id);

    if (!contact) {
      return res.status(404).json({
        success: false,
        error: "Contact not found"
      });
    }

    await Contact.findByIdAndDelete(id);
    res.status(200).json({
      success: true,
      message: "Contact deleted successfully"
    });

  } catch (error) {
    console.error("Error deleting contact:", error);
    res.status(500).json({
      success: false,
      error: "Failed to delete contact",
      details: error.message
    });
  }
};

// Create News


// Removed duplicate multer storage - using middleware/multer.js instead






exports.createNews = async (req, res) => {
  try {
    if (!req.file) {
      return res.status(400).json({
        success: false,
        message: "Image is required"
      });
    }

    const {
      title,
      content,
      moreContent,
      author,
      category,
      tags,
```

```
            featured,
            published,
            readTime,
            metaDescription
          } = req.body;

          if (!title || !content || !moreContent) {
            return res.status(400).json({
              success: false,
              message: "Title, content, and full content are required"
            });
          }

          // Validate file buffer exists
          if (!req.file.buffer || req.file.buffer.length === 0) {
            return res.status(400).json({
              success: false,
              message: "Invalid image file. Please upload a valid image."
            });
          }

          // Upload image to Cloudinary with error handling
          let cloudinaryResult;
          try {
            cloudinaryResult = await uploadBufferToCloudinary(
              req.file.buffer,
              'itu/blog-posts',
              `blog-${Date.now()}`
            );

            if (!cloudinaryResult || !cloudinaryResult.url) {
              throw new Error('Cloudinary upload failed - no URL returned');
            }
          } catch (cloudinaryError) {
            console.error("Cloudinary upload error:", cloudinaryError);
            return res.status(500).json({
              success: false,
              message: "Failed to upload image to Cloudinary",
              error: cloudinaryError.message
            });
          }

          // Parse tags if it's a string
          let parsedTags = [];
          if (tags) {
            parsedTags = typeof tags === 'string' ? tags.split(',').map(tag => tag.trim()).filter(tag => tag)
      : tags;
          }

          const news = new News({
            title,
            content,
            moreContent,
            image: cloudinaryResult.url, // Store Cloudinary URL
            cloudinaryPublicId: cloudinaryResult.public_id, // Store public_id for deletion
            author: author || "ITU Admin",
            category: category || "News",
            tags: parsedTags,
            featured: featured === 'true' || featured === true,
            published: published !== 'false' && published !== false,
            readTime: readTime ? parseInt(readTime) : 5,
            metaDescription: metaDescription || content.substring(0, 160)
          });

          await news.save();
          res.status(201).json({
            success: true,
            message: "Blog post created successfully",
            news
          });
        } catch (error) {
          console.error("Create News Error:", error);
          res.status(500).json({
            success: false,
            message: "Failed to create blog post",
```

```
      error: error.message
    });
  }
};




exports.getAllNews = async (req, res) => {
  try {
    const news = await News.find().sort({ createdAt: -1 });

    if (!news.length) {
      return res.status(404).json({
        success: false,
        message: "No news articles found"
      });
    }

    const updatedNews = news.map(item => {
      let imageUrl = "/default-image.png"; // Default fallback

      if (item.image) {
        // If image is already a full URL (Cloudinary, http/https, or data URI), use it directly
        if (/^(https?|data):/i.test(item.image)) {
          imageUrl = item.image;
        }
        // Legacy: If it's a relative path or local filename, construct URL (for backward
compatibility)
        else {
          const baseUrl = `${req.protocol}://${req.get("host")}/`;
          imageUrl = `${baseUrl}${item.image.replace(/^\//, '')}`; // Remove leading slash if present
        }
      }

      return {
        ...item._doc,
        image: imageUrl
      };
    });

    res.status(200).json({
      success: true,
      message: "News articles retrieved successfully",
      news: updatedNews
    });
  } catch (error) {
    console.error("Database error:", error);
    res.status(500).json({
      success: false,
      message: "Failed to retrieve news"
    });
  }
};




exports.editNews = async (req, res) => {
  try {
    const { id } = req.params;
    const {
      title,
      content,
      moreContent,
      author,
      category,
      tags,
      featured,
      published,
      readTime,
      metaDescription
    } = req.body;

    const news = await News.findById(id);
```

```
        if (!news) {
          return res.status(404).json({
            success: false,
            message: "News not found"
          });
        }

        // Update text fields
        if (title) news.title = title;
        if (content) news.content = content;
        if (moreContent) news.moreContent = moreContent;
        if (author) news.author = author;
        if (category) news.category = category;
        if (readTime) news.readTime = parseInt(readTime);
        if (metaDescription !== undefined) news.metaDescription = metaDescription;

        // Handle boolean fields
        if (featured !== undefined) {
          news.featured = featured === 'true' || featured === true;
        }
        if (published !== undefined) {
          news.published = published !== 'false' && published !== false;
        }

        // Parse and update tags
        if (tags !== undefined) {
          if (typeof tags === 'string') {
            news.tags = tags.split(',').map(tag => tag.trim()).filter(tag => tag);
          } else if (Array.isArray(tags)) {
            news.tags = tags;
          }
        }

        // If new image uploaded, upload to Cloudinary
        if (req.file) {
          // Validate file buffer exists
          if (!req.file.buffer || req.file.buffer.length === 0) {
            return res.status(400).json({
              success: false,
              message: "Invalid image file. Please upload a valid image."
            });
          }

          // Delete old image from Cloudinary if it exists
          if (news.cloudinaryPublicId) {
            try {
              await deleteFromCloudinary(news.cloudinaryPublicId);
            } catch (error) {
              console.error("Error deleting old Cloudinary image:", error);
              // Continue even if deletion fails
            }
          }

          // Upload new image to Cloudinary with error handling
          let cloudinaryResult;
          try {
            cloudinaryResult = await uploadBufferToCloudinary(
              req.file.buffer,
              'itu/blog-posts',
              `blog-${Date.now()}`
            );

            if (!cloudinaryResult || !cloudinaryResult.url) {
              throw new Error('Cloudinary upload failed - no URL returned');
            }

            news.image = cloudinaryResult.url;
            news.cloudinaryPublicId = cloudinaryResult.public_id;
          } catch (cloudinaryError) {
            console.error("Cloudinary upload error:", cloudinaryError);
            return res.status(500).json({
              success: false,
              message: "Failed to upload image to Cloudinary",
              error: cloudinaryError.message
            });
```

```
        }
      }

      await news.save();
      res.status(200).json({
        success: true,
        message: "News updated successfully",
        news
      });
    } catch (err) {
      console.error("Edit News Error:", err);
      res.status(500).json({
        success: false,
        message: "Update failed",
        error: err.message
      });
    }
  };

  // DELETE News
  exports.deleteNews = async (req, res) => {
    try {
      const { id } = req.params;
      const news = await News.findById(id);

      if (!news) {
        return res.status(404).json({ success: false, message: "News not found" });
      }

      // Delete image from Cloudinary if it exists
      if (news.cloudinaryPublicId) {
        try {
          await deleteFromCloudinary(news.cloudinaryPublicId);
        } catch (error) {
          console.error("Error deleting from Cloudinary:", error);
          // Continue with database deletion even if Cloudinary deletion fails
        }
      }

      await News.findByIdAndDelete(id);
      res.status(200).json({ success: true, message: "News deleted successfully" });

    } catch (err) {
      console.error("Delete News Error:", err);
      res.status(500).json({
        success: false,
        message: "Delete failed",
        error: err.message // Send detailed error in development
      });
    }
  };



  //HomePage Slider mngment




  // POST: Upload a new slider image with Cloudinary
  exports.createSlider = async (req, res) => {
    try {
      if (!req.file) {
        return res.status(400).json({
          success: false,
          message: "No image uploaded"
        });
      }

      // Upload to Cloudinary
      const cloudinaryResult = await uploadBufferToCloudinary(
        req.file.buffer,
        'itu/sliders',
        `slider-${Date.now()}`
      );
```

```
    // Save Cloudinary URL to database
    const newSlider = new Slider({
      filename: cloudinaryResult.url, // Store Cloudinary URL instead of filename
      cloudinaryPublicId: cloudinaryResult.public_id, // Store public_id for deletion
      uploadedAt: new Date()
    });

    await newSlider.save();

    res.status(201).json({
      success: true,
      message: "Slider uploaded successfully to Cloudinary",
      slider: newSlider
    });
  } catch (error) {
    console.error("Slider upload error:", error);
    res.status(500).json({
      success: false,
      message: "Failed to upload slider",
      error: error.message
    });
  }
};



// GET: Get all slider images
exports.getSliders = async (req, res) => {
  try {
    const sliders = await Slider.find().sort({ uploadedAt: -1 });
    res.status(200).json(sliders);
  } catch (error) {
    console.error("Fetch error:", error);
    res.status(500).json({ success: false, message: "Internal server error" });
  }
};

// PUT: Update a slider image by replacing the image
exports.updateSlider = async (req, res) => {
  try {
    const { id } = req.params;
    const slider = await Slider.findById(id);

    if (!slider) return res.status(404).json({ success: false, message: "Slider not found" });

    if (!req.file) {
      return res.status(400).json({ success: false, message: "No image uploaded" });
    }

    // Delete old image from Cloudinary if it exists
    if (slider.cloudinaryPublicId) {
      try {
        await deleteFromCloudinary(slider.cloudinaryPublicId);
      } catch (error) {
        console.error("Error deleting old Cloudinary image:", error);
        // Continue even if deletion fails
      }
    }

    // Upload new image to Cloudinary
    const cloudinaryResult = await uploadBufferToCloudinary(
      req.file.buffer,
      'itu/sliders',
      `slider-${Date.now()}`
    );

    // Update slider with new Cloudinary URL
    slider.filename = cloudinaryResult.url;
    slider.cloudinaryPublicId = cloudinaryResult.public_id;
    slider.uploadedAt = new Date();
    await slider.save();

    res.status(200).json({ success: true, message: "Slider updated", slider });
  } catch (error) {
```

```
    console.error("Update error:", error);
    res.status(500).json({ success: false, message: "Internal server error", error: error.message });
  }
};

// DELETE: Delete a slider image from Cloudinary
exports.deleteSlider = async (req, res) => {
  try {
    const { id } = req.params;
    const slider = await Slider.findById(id);

    if (!slider) {
      return res.status(404).json({ success: false, message: "Slider not found" });
    }

    // Delete from Cloudinary if public_id exists
    if (slider.cloudinaryPublicId) {
      try {
        await deleteFromCloudinary(slider.cloudinaryPublicId);
      } catch (error) {
        console.error("Error deleting from Cloudinary:", error);
        // Continue with database deletion even if Cloudinary deletion fails
      }
    }

    // Delete from database
    await Slider.findByIdAndDelete(id);

    res.json({ success: true, message: "Slider deleted successfully from Cloudinary and database" });
  } catch (error) {
    console.error("Delete error:", error);
    res.status(500).json({ success: false, message: "Server error", error: error.message });
  }
};




//Form
exports.getForm = async (req, res) => {
  try {
    const forms = await AccelerationForm.find().sort({ createdAt: -1 });
    res.status(200).json(forms);
  } catch (error) {
    res.status(500).json({ error: "Failed to fetch forms" });
  }
};




//GalleryMngment

exports.createGallery = async (req, res) => {
  try {
    if (!req.file) {
      return res.status(400).json({
        success: false,
        message: "No image uploaded"
      });
    }

    // Upload to Cloudinary
    const cloudinaryResult = await uploadBufferToCloudinary(
      req.file.buffer,
      'itu/gallery',
      `gallery-${Date.now()}`
    );

    // Save Cloudinary URL to database
    const newGallery = new Gallery({
      filename: cloudinaryResult.url, // Store Cloudinary URL instead of filename
      cloudinaryPublicId: cloudinaryResult.public_id, // Store public_id for deletion
```

```
      title: req.body.title || null, // Optional title
      uploadedAt: new Date()
    });

    await newGallery.save();

    res.status(201).json({
      success: true,
      message: "Gallery image uploaded successfully to Cloudinary",
      gallery: newGallery
    });
  } catch (error) {
    console.error("Gallery upload error:", error);
    res.status(500).json({
      success: false,
      message: "Failed to upload gallery image",
      error: error.message
    });
  }
};



// GET: Get all slider images
exports.getGallery = async (req, res) => {
  try {
    const Gallerys = await Gallery.find().sort({ uploadedAt: -1 });
    res.status(200).json(Gallerys);
  } catch (error) {
    console.error("Fetch error:", error);
    res.status(500).json({ success: false, message: "Internal server error" });
  }
};

// PUT: Update a gallery image by replacing the image
exports.updateGallery = async (req, res) => {
  try {
    const { id } = req.params;
    const gallery = await Gallery.findById(id);

    if (!gallery) return res.status(404).json({ success: false, message: "Gallery image not found" });

    if (!req.file) {
      return res.status(400).json({ success: false, message: "No image uploaded" });
    }

    // Delete old image from Cloudinary if it exists
    if (gallery.cloudinaryPublicId) {
      try {
        await deleteFromCloudinary(gallery.cloudinaryPublicId);
      } catch (error) {
        console.error("Error deleting old Cloudinary image:", error);
        // Continue even if deletion fails
      }
    }

    // Upload new image to Cloudinary
    const cloudinaryResult = await uploadBufferToCloudinary(
      req.file.buffer,
      'itu/gallery',
      `gallery-${Date.now()}`
    );

    // Update gallery with new Cloudinary URL
    gallery.filename = cloudinaryResult.url;
    gallery.cloudinaryPublicId = cloudinaryResult.public_id;
    if (req.body.title) gallery.title = req.body.title;
    gallery.uploadedAt = new Date();
    await gallery.save();

    res.status(200).json({ success: true, message: "Gallery image updated", gallery });
  } catch (error) {
    console.error("Update error:", error);
    res.status(500).json({ success: false, message: "Internal server error", error: error.message });
  }
```

```
};

// DELETE: Delete a gallery image from Cloudinary
exports.deleteGallery = async (req, res) => {
  try {
    const { id } = req.params;
    const gallery = await Gallery.findById(id);

    if (!gallery) {
      return res.status(404).json({ success: false, message: "Gallery image not found" });
    }

    // Delete from Cloudinary if public_id exists
    if (gallery.cloudinaryPublicId) {
      try {
        await deleteFromCloudinary(gallery.cloudinaryPublicId);
      } catch (error) {
        console.error("Error deleting from Cloudinary:", error);
        // Continue with database deletion even if Cloudinary deletion fails
      }
    }

    // Delete from database
    await Gallery.findByIdAndDelete(id);

    res.json({ success: true, message: "Gallery image deleted successfully from Cloudinary and
database" });
  } catch (error) {
    console.error("Delete error:", error);
    res.status(500).json({ success: false, message: "Server error" });
  }
};

// ========== SELF DEFENCE SLIDER MANAGEMENT ==========

// POST: Upload a new self defence slider image with Cloudinary
exports.createSelfDefenceSlider = async (req, res) => {
  try {
    if (!req.file) {
      return res.status(400).json({
        success: false,
        message: "No image uploaded"
      });
    }

    // Upload to Cloudinary
    const cloudinaryResult = await uploadBufferToCloudinary(
      req.file.buffer,
      'itu/self-defence-sliders',
      `self-defence-slider-${Date.now()}`
    );

    // Save Cloudinary URL to database
    const newSlider = new SelfDefenceSlider({
      filename: cloudinaryResult.url, // Store Cloudinary URL instead of filename
      cloudinaryPublicId: cloudinaryResult.public_id, // Store public_id for deletion
      uploadedAt: new Date()
    });

    await newSlider.save();

    res.status(201).json({
      success: true,
      message: "Self Defence slider uploaded successfully to Cloudinary",
      slider: newSlider
    });
  } catch (error) {
    console.error("Self Defence Slider upload error:", error);
    res.status(500).json({
      success: false,
      message: "Failed to upload self defence slider",
      error: error.message
    });
  }
};
```

```
// GET: Get all self defence slider images
exports.getSelfDefenceSliders = async (req, res) => {
  try {
    const sliders = await SelfDefenceSlider.find().sort({ uploadedAt: -1 });
    res.status(200).json(sliders);
  } catch (error) {
    console.error("Fetch error:", error);
    res.status(500).json({ success: false, message: "Internal server error" });
  }
};

// PUT: Update a self defence slider image by replacing the image
exports.updateSelfDefenceSlider = async (req, res) => {
  try {
    const { id } = req.params;
    const slider = await SelfDefenceSlider.findById(id);

    if (!slider) return res.status(404).json({ success: false, message: "Self Defence slider not found"
});

    if (!req.file) {
      return res.status(400).json({ success: false, message: "No image uploaded" });
    }

    // Delete old image from Cloudinary if it exists
    if (slider.cloudinaryPublicId) {
      try {
        await deleteFromCloudinary(slider.cloudinaryPublicId);
      } catch (error) {
        console.error("Error deleting old Cloudinary image:", error);
        // Continue even if deletion fails
      }
    }

    // Upload new image to Cloudinary
    const cloudinaryResult = await uploadBufferToCloudinary(
      req.file.buffer,
      'itu/self-defence-sliders',
      `self-defence-slider-${Date.now()}`
    );

    // Update slider with new Cloudinary URL
    slider.filename = cloudinaryResult.url;
    slider.cloudinaryPublicId = cloudinaryResult.public_id;
    slider.uploadedAt = new Date();
    await slider.save();

    res.status(200).json({ success: true, message: "Self Defence slider updated", slider });
  } catch (error) {
    console.error("Update error:", error);
    res.status(500).json({ success: false, message: "Internal server error", error: error.message });
  }
};

// DELETE: Delete a self defence slider image from Cloudinary
exports.deleteSelfDefenceSlider = async (req, res) => {
  try {
    const { id } = req.params;
    const slider = await SelfDefenceSlider.findById(id);

    if (!slider) {
      return res.status(404).json({ success: false, message: "Self Defence slider not found" });
    }

    // Delete from Cloudinary if public_id exists
    if (slider.cloudinaryPublicId) {
      try {
        await deleteFromCloudinary(slider.cloudinaryPublicId);
      } catch (error) {
        console.error("Error deleting from Cloudinary:", error);
        // Continue with database deletion even if Cloudinary deletion fails
      }
    }
```

```
    // Delete from database
    await SelfDefenceSlider.findByIdAndDelete(id);

    res.json({ success: true, message: "Self Defence slider deleted successfully from Cloudinary and
database" });
  } catch (error) {
    console.error("Delete error:", error);
    res.status(500).json({ success: false, message: "Server error", error: error.message });
  }
};

// ========== CATEGORY SLIDER MANAGEMENT ==========

// POST: Upload a new category slider image with Cloudinary
exports.createCategorySlider = async (req, res) => {
  try {
    if (!req.file) {
```

PART 2: LAST 25 PAGES (or equivalent)

```
                </div>

                <div>
                  <label className="block text-sm font-medium text-gray-700 mb-1">Logo</label>
                  <input
                    type="file"
                    onChange={handleLogoUpload}
                    accept="image/*"
                    className="w-full px-3 py-2 border border-gray-300 rounded-md"
                  />
                  <div className="mt-2">
                    {logoPreview ? (
                      <img
                        src={logoPreview}
                        alt="Logo Preview"
                        className="h-32 w-auto rounded-lg object-cover border-2 border-blue-500 shadow-
md"
                      />
                    ) : profileData?.logo ? (
                      <img
                        src={getImageUrl(profileData.logo)}
                      alt="Union Logo"
                        className="h-32 w-auto rounded-lg object-cover border-2 border-gray-300"
                        onError={(e) => {
                          console.error('Logo image error:', e.target.src);
                          e.target.style.display = 'none';
                        }}
                      />
                    ) : null}
                  </div>
                </div>

                <div>
                  <label className="block text-sm font-medium text-gray-700 mb-1">
{profileData?.district ? 'Secretary Image' : 'General Secretary Image'}</label>
                  <input
                    type="file"
                    onChange={handleGeneralSecretaryImageUpload}
                    accept="image/*"
                    className="w-full px-3 py-2 border border-gray-300 rounded-md"
                  />
                  <div className="mt-2">
                    {secretaryImagePreview ? (
                      <img
                        src={secretaryImagePreview}
                        alt={profileData?.district ? "Secretary Preview" : "General Secretary Preview"}
                        className="h-32 w-auto rounded-lg object-cover border-2 border-blue-500 shadow-
md"
                      />
                    ) : profileData?.generalSecretaryImage ? (
                      <img
                        src={getImageUrl(profileData.generalSecretaryImage)}
                        alt={profileData?.district ? "Secretary" : "General Secretary"}
                        className="h-32 w-auto rounded-lg object-cover border-2 border-gray-300"
                        onError={(e) => {
                          console.error('Secretary image error:', e.target.src);
                          e.target.style.display = 'none';
                        }}
                      />
                    ) : null}
                  </div>
                </div>

                <div className="flex gap-3">
                <button
                  type="submit"
                  className="px-4 py-2 bg-blue-600 text-white rounded-md hover:bg-blue-700"
                >
```

```
                    Update Profile
                  </button>
                    <button
                      type="button"
                      onClick={() => setIsEditing(false)}
                      className="px-4 py-2 bg-gray-500 text-white rounded-md hover:bg-gray-600"
                    >
                      Cancel
                    </button>
                  </div>
                </form>
              ) : (
                <div className="space-y-6">
                  <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
                    <div className="border-b pb-4">
                      <p className="text-sm font-medium text-gray-500 mb-1">State</p>
                      <p className="text-lg font-semibold">{profileData?.state || 'N/A'}</p>
                    </div>

                    <div className="border-b pb-4">
                      <p className="text-sm font-medium text-gray-500 mb-1">District</p>
                      <p className="text-lg font-semibold">{profileData?.district || 'N/A'}</p>
                    </div>

                    <div className="border-b pb-4">
                      <p className="text-sm font-medium text-gray-500 mb-1">Organization Name</p>
                      <p className="text-lg font-semibold">{profileData?.name || 'N/A'}</p>
                    </div>

                    <div className="border-b pb-4">
                      <p className="text-sm font-medium text-gray-500 mb-1">Phone</p>
                      <p className="text-lg font-semibold">{profileData?.phone || 'N/A'}</p>
                    </div>

                    <div className="border-b pb-4">
                      <p className="text-sm font-medium text-gray-500 mb-1">Address</p>
                      <p className="text-lg font-semibold">{profileData?.address || 'N/A'}</p>
                    </div>

                    <div className="border-b pb-4">
                      <p className="text-sm font-medium text-gray-500 mb-1">{profileData?.district ?
'Secretary Name' : 'General Secretary Name'}</p>
                      <p className="text-lg font-semibold">{profileData?.secretaryName || 'N/A'}</p>
                    </div>

                    <div className="border-b pb-4">
                      <p className="text-sm font-medium text-gray-500 mb-1">President Name</p>
                      <p className="text-lg font-semibold">{profileData?.presidentName || 'N/A'}</p>
                    </div>

                    <div className="border-b pb-4">
                      <p className="text-sm font-medium text-gray-500 mb-1">Established Year</p>
                      <p className="text-lg font-semibold">
                        {profileData?.establishedDate ? new
Date(profileData.establishedDate).getFullYear() : 'N/A'}
                      </p>
                    </div>

                    <div className="border-b pb-4">
                      <p className="text-sm font-medium text-gray-500 mb-1">Status</p>
                      <span className={`px-2 inline-flex text-xs leading-5 font-semibold rounded-full
${profileData?.status === 'approved' ? 'bg-green-100 text-green-800' : 'bg-yellow-100 text-yellow-
800'}`}>
                        {profileData?.status?.toUpperCase() || 'N/A'}
                      </span>
                    </div>
                  </div>

                  {profileData?.logo && (
                    <div>
                      <p className="text-sm font-medium text-gray-500 mb-2">Logo</p>
                      <img
                        src={getImageUrl(profileData.logo)}
                        alt="Union Logo"
                        className="h-32 rounded-lg shadow-md object-cover"
```

```
                        onError={(e) => {
                          e.target.style.display = 'none';
                        }}
                      />
                    </div>
                  )}

                  {profileData?.generalSecretaryImage && (
                    <div>
                      <p className="text-sm font-medium text-gray-500 mb-2">{profileData?.district ?
'Secretary Image' : 'General Secretary Image'}</p>
                      <img
                        src={getImageUrl(profileData.generalSecretaryImage)}
                        alt={profileData?.district ? "Secretary" : "General Secretary"}
                        className="h-32 rounded-lg shadow-md object-cover"
                        onError={(e) => {
                          e.target.style.display = 'none';
                        }}
                      />
                    </div>
                  )}
                </div>
              )}
            </div>
          );

        case 'players':
          return (
            <div className="bg-white rounded-lg shadow p-6">
              <div className="flex flex-col md:flex-row md:items-center md:justify-between mb-6">
                <h2 className="text-2xl font-bold mb-4 md:mb-0">Approved Players</h2>
                <div className="relative w-full md:w-64">
                  <Search className="absolute left-3 top-1/2 transform -translate-y-1/2 text-gray-400"
size={20} />
                  <input
                    type="text"
                    placeholder="Search by name, email, phone, belt..."
                    value={searchTerm}
                    onChange={(e) => {
                      setSearchTerm(e.target.value);
                      setCurrentPage(1); // Reset to first page on search
                    }}
                    className="w-full pl-10 pr-4 py-2 border border-gray-300 rounded-lg focus:ring-2
focus:ring-blue-500 focus:border-transparent"
                  />
                </div>
              </div>

              {loadingPlayers ? (
                <div className="flex items-center justify-center py-12">
                  <div className="animate-spin rounded-full h-8 w-8 border-t-2 border-b-2 border-blue-
500"></div>
                  <span className="ml-3 text-gray-600">Loading players...</span>
                </div>
              ) : players.length === 0 ? (
                <div className="text-center py-12">
                  <User size={48} className="mx-auto text-gray-400 mb-4" />
                  <p className="text-gray-600">
                    {searchTerm ? 'No players found matching your search' : 'No approved players yet'}
                  </p>
                </div>
              ) : (
                <>
                  <div className="mb-4 text-sm text-gray-600">
                    Showing {((currentPage - 1) * 10) + 1} - {Math.min(currentPage * 10,
pagination.totalPlayers)} of {pagination.totalPlayers} players
                  </div>
                <div className="overflow-x-auto">
                  <table className="min-w-full divide-y divide-gray-200">
                    <thead className="bg-gray-50">
                      <tr>
                          <th className="px-4 py-3 text-left text-xs font-medium text-gray-500 uppercase
tracking-wider">Name</th>
                          <th className="px-4 py-3 text-left text-xs font-medium text-gray-500 uppercase
tracking-wider">Player ID</th>
```

```
                        <th className="px-4 py-3 text-left text-xs font-medium text-gray-500 uppercase
tracking-wider">Email</th>
                        <th className="px-4 py-3 text-left text-xs font-medium text-gray-500 uppercase
tracking-wider">Phone</th>
                        <th className="px-4 py-3 text-left text-xs font-medium text-gray-500 uppercase
tracking-wider">Password</th>
                        <th className="px-4 py-3 text-left text-xs font-medium text-gray-500 uppercase
tracking-wider">Belt Level</th>
                        <th className="px-4 py-3 text-left text-xs font-medium text-gray-500 uppercase
tracking-wider">Experience</th>
                        <th className="px-4 py-3 text-left text-xs font-medium text-gray-500 uppercase
tracking-wider">DOB</th>
                      </tr>
                    </thead>
                    <tbody className="bg-white divide-y divide-gray-200">
                      {players.map(player => (
                        <tr key={player._id} className="hover:bg-gray-50">
                          <td className="px-4 py-4 whitespace-nowrap">
                            <div className="flex items-center">
                              {player.photo ? (
                                <img
                                  src={player.photo.startsWith('http') ? player.photo :
`${import.meta.env.VITE_API_BASE_URL || 'http://localhost:3001'}/${player.photo}`}
                                  alt={player.name}
                                  className="h-10 w-10 rounded-full mr-3 object-cover"
                                  onError={(e) => {
                                    e.target.style.display = 'none';
                                  }}
                                />
                              ) : (
                                <div className="h-10 w-10 rounded-full bg-gray-200 flex items-center
justify-center mr-3">
                                  <User size={20} className="text-gray-400" />
                                </div>
                              )}
                              <span className="text-sm font-medium text-gray-900">{player.name}</span>
                            </div>
                          </td>
                          <td className="px-4 py-4 whitespace-nowrap text-sm text-gray-500">
{player.playerId || 'N/A'}</td>
                          <td className="px-4 py-4 whitespace-nowrap text-sm text-gray-500">
{player.email}</td>
                          <td className="px-4 py-4 whitespace-nowrap text-sm text-gray-500">
{player.phone}</td>
                          <td className="px-4 py-4 whitespace-nowrap">
                            {player.password ? (
                              <span className="text-sm font-mono bg-gray-100 px-2 py-1 rounded text-
gray-700">
                                {player.password}
                              </span>
                            ) : (
                              <span className="text-sm text-gray-400">N/A</span>
                            )}
                          </td>
                          <td className="px-4 py-4 whitespace-nowrap">
                            <span className="px-2 inline-flex text-xs leading-5 font-semibold rounded-
full bg-blue-100 text-blue-800">
                              {player.beltLevel}
                          </span>
                        </td>
                          <td className="px-4 py-4 whitespace-nowrap text-sm text-gray-500">
{player.yearsOfExperience} years</td>
                          <td className="px-4 py-4 whitespace-nowrap text-sm text-gray-500">
                            {player.dob ? new Date(player.dob).toLocaleDateString() : 'N/A'}
                        </td>
                      </tr>
                    ))}
                    </tbody>
                  </table>
                </div>

                  {/* Pagination */}
                  {pagination.totalPages > 1 && (
                    <div className="flex items-center justify-between mt-6 pt-4 border-t border-gray-
200">
```

```
                    <div className="text-sm text-gray-700">
                      Page {pagination.currentPage} of {pagination.totalPages}
                    </div>
                    <div className="flex items-center gap-2">
                      <button
                        onClick={() => setCurrentPage(prev => Math.max(1, prev - 1))}
                        disabled={currentPage === 1}
                        className="px-4 py-2 border border-gray-300 rounded-md text-sm font-medium
text-gray-700 bg-white hover:bg-gray-50 disabled:opacity-50 disabled:cursor-not-allowed flex items-
center gap-1"
                      >
                        <ChevronLeft size={16} />
                        Previous
                      </button>
                      <button
                        onClick={() => setCurrentPage(prev => Math.min(pagination.totalPages, prev +
1))}
                        disabled={currentPage === pagination.totalPages}
                        className="px-4 py-2 border border-gray-300 rounded-md text-sm font-medium
text-gray-700 bg-white hover:bg-gray-50 disabled:opacity-50 disabled:cursor-not-allowed flex items-
center gap-1"
                      >
                        Next
                        <ChevronRight size={16} />
                      </button>
                    </div>
                  </div>
                )}
              </>
            )}
          </div>
        );

      case 'achievements':
        return (
          <div className="bg-white rounded-lg shadow p-6">
            <h2 className="text-2xl font-bold mb-6">Achievements</h2>
            <div className="border border-gray-200 rounded-md p-4">
              <p>Achievement management feature will be available soon</p>
            </div>
          </div>
        );

      case 'belt-promotion':
        return (
          <BeltPromotionForm unionId={userId} />
        );

      case 'competition':
        return (
          <CompetitionRegistration unionId={userId} />
        );

      case 'events':
        return (
          <div className="bg-white rounded-lg shadow p-6">
            <h2 className="text-2xl font-bold mb-6">Events & News</h2>

            {loadingNews ? (
              <div className="flex items-center justify-center py-12">
                <div className="animate-spin rounded-full h-8 w-8 border-t-2 border-b-2 border-blue-
500"></div>
                <span className="ml-3 text-gray-600">Loading events...</span>
              </div>
            ) : selectedNews ? (
              <div>
                <button
                  onClick={() => setSelectedNews(null)}
                  className="mb-4 flex items-center gap-2 text-blue-600 hover:text-blue-800"
                >
                  <ChevronLeft size={20} />
                  Back to Events
                </button>
                <div className="relative w-full h-64 bg-gray-200 rounded-lg overflow-hidden mb-6">
                  <img
```

```
                    src={getImageUrl(selectedNews.image)}
                    alt={selectedNews.title}
                    className="w-full h-full object-cover"
                    onError={(e) => {
                      e.target.style.display = 'none';
                    }}
                  />
                </div>
                <h3 className="text-2xl font-bold text-gray-900 mb-4">{selectedNews.title}</h3>
                <p className="text-gray-700 font-semibold mb-4">{selectedNews.content}</p>
                {selectedNews.moreContent && (
                  <div className="text-gray-600 whitespace-pre-line">{selectedNews.moreContent}</div>
                )}
                {selectedNews.author && (
                  <p className="text-sm text-gray-500 mt-4">By {selectedNews.author}</p>
                )}
              </div>
            ) : newsList.length > 0 ? (
              <>
                {newsList.length > 0 && (
                  <div
                    className="relative bg-white rounded-xl shadow-md overflow-hidden cursor-pointer
hover:shadow-lg transition-shadow mb-6"
                    onClick={() => setSelectedNews(newsList[0])}
                  >
                    <div className="relative w-full h-64 overflow-hidden">
                      <img
                        src={getImageUrl(newsList[0].image)}
                        alt="Featured News"
                        className="w-full h-full object-cover"
                        onError={(e) => {
                          e.target.style.display = 'none';
                        }}
                      />
                    </div>
                    <div className="absolute inset-0 bg-gradient-to-t from-black/70 via-black/30 to-
transparent flex flex-col justify-end p-6">
                      <h3 className="text-white text-2xl font-bold mb-2">{newsList[0].title}</h3>
                      <p className="text-gray-200">
                        {newsList[0].content.substring(0, 150)}...
                      </p>
                    </div>
                  </div>
                )}

                <div className="grid md:grid-cols-2 lg:grid-cols-3 gap-6">
                  {newsList.slice(1).map((news) => (
                    <div
                      key={news._id}
                      className="bg-white rounded-xl shadow-md overflow-hidden cursor-pointer
hover:shadow-lg transition-all"
                      onClick={() => setSelectedNews(news)}
                    >
                      <div className="relative w-full h-40 bg-gray-200 overflow-hidden">
                        <img
                          src={getImageUrl(news.image)}
                          alt={news.title}
                          className="w-full h-full object-cover"
                          onError={(e) => {
                            e.target.style.display = 'none';
                          }}
                        />
                      </div>
                      <div className="p-4">
                        <h4 className="text-lg font-semibold text-gray-900 mb-2 line-clamp-2">
{news.title}</h4>
                        <p className="text-gray-600 text-sm line-clamp-3">
                          {news.content.substring(0, 100)}...
                        </p>
                        <button className="mt-4 text-blue-600 hover:text-blue-800 font-medium text-sm">
                          Read More →
                        </button>
            </div>
                    </div>
                  ))}
```

```
                  </div>
                </>
              ) : (
                <div className="text-center py-12">
                  <p className="text-gray-600">No events or news available at the moment.</p>
                </div>
              )}
            </div>
          );

        default:
          return (
            <div className="bg-white rounded-lg shadow p-6">
              <h2 className="text-2xl font-bold mb-6">Dashboard Overview</h2>
              {loading ? (
                <p>Loading...</p>
              ) : profileData ? (
                <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
                  <div className="border-b pb-2">
                    <p className="text-sm font-medium text-gray-500">Union Name</p>
                    <p className="text-lg">{profileData.name}</p>
                  </div>
                  <div className="border-b pb-2">
                    <p className="text-sm font-medium text-gray-500">State</p>
                    <p className="text-lg">{profileData.state}</p>
                  </div>
                  <div className="border-b pb-2">
                    <p className="text-sm font-medium text-gray-500">Status</p>
                    <span className={`px-2 inline-flex text-xs leading-5 font-semibold rounded-full
${profileData.status === 'approved' ? 'bg-green-100 text-green-800' : 'bg-yellow-100 text-yellow-
800'}`}>
                      {profileData.status.toUpperCase()}
                    </span>
                  </div>
                  <div className="border-b pb-2">
                    <p className="text-sm font-medium text-gray-500">President</p>
                    <p className="text-lg">{profileData.presidentName || 'N/A'}</p>
                  </div>
                  <div className="border-b pb-2">
                    <p className="text-sm font-medium text-gray-500">Established Year</p>
                    <p className="text-lg">
                      {profileData.establishedDate ? new Date(profileData.establishedDate).getFullYear()
: 'N/A'}
                    </p>
                  </div>
                </div>
              ) : (
                <p>No data available</p>
              )}
            </div>
          );
      }
    };

    return (
      <div className="flex h-screen bg-gray-100">
        {/* Sidebar */}
        <div className={`bg-blue-800 text-white transition-all duration-300 ${collapsed ? 'w-20' : 'w-
64'} flex flex-col relative`}>
          <div className={`h-16 flex items-center justify-center border-b border-blue-700 ${collapsed ?
'text-xl' : 'text-2xl'} font-bold`}>
            {collapsed ? 'UM' : 'Union member'}
          </div>
          <nav className="flex-1 overflow-y-auto">
            <ul className="space-y-1 p-2">
              <li>
                <button
                  onClick={() => setActiveMenu('dashboard')}
                  className={`w-full flex items-center p-3 rounded-md ${activeMenu === 'dashboard' ? 'bg-
blue-700' : 'hover:bg-blue-700'}`}
                >
                  <span className="mr-3">🏠</span>
                  {!collapsed && <span>Dashboard</span>}
                </button>
              </li>
```

```
              <li>
                <button
                  onClick={() => setActiveMenu('profile')}
                  className={`w-full flex items-center p-3 rounded-md ${activeMenu === 'profile' ? 'bg-
blue-700' : 'hover:bg-blue-700'}`}
                >
                  <span className="mr-3">👤</span>
                  {!collapsed && <span>Update Profile</span>}
                </button>
              </li>
              <li>
                <button
                  onClick={() => setActiveMenu('players')}
                  className={`w-full flex items-center p-3 rounded-md ${activeMenu === 'players' ? 'bg-
blue-700' : 'hover:bg-blue-700'}`}
                >
                  <span className="mr-3">👥</span>
                  {!collapsed && <span>Approved Players</span>}
                </button>
              </li>
              <li>
                <button
                  onClick={() => setActiveMenu('achievements')}
                  className={`w-full flex items-center p-3 rounded-md ${activeMenu === 'achievements' ?
'bg-blue-700' : 'hover:bg-blue-700'}`}
                >
                  <span className="mr-3">🏆</span>
                  {!collapsed && <span>Achievements</span>}
                </button>
              </li>
              <li>
                <button
                  onClick={() => setActiveMenu('events')}
                  className={`w-full flex items-center p-3 rounded-md ${activeMenu === 'events' ? 'bg-
blue-700' : 'hover:bg-blue-700'}`}
                >
                  <span className="mr-3">📅</span>
                  {!collapsed && <span>Events</span>}
                </button>
              </li>
              <li>
                <button
                  onClick={() => setActiveMenu('belt-promotion')}
                  className={`w-full flex items-center p-3 rounded-md ${activeMenu === 'belt-promotion' ?
'bg-blue-700' : 'hover:bg-blue-700'}`}
                >
                  <span className="mr-3">🎯</span>
                  {!collapsed && <span>Belt Promotion</span>}
                </button>
              </li>
              <li>
                <button
                  onClick={() => setActiveMenu('competition')}
                  className={`w-full flex items-center p-3 rounded-md ${activeMenu === 'competition' ?
'bg-blue-700' : 'hover:bg-blue-700'}`}
                >
                  <span className="mr-3">🏆</span>
                  {!collapsed && <span>Competition</span>}
                </button>
              </li>
            </ul>
          </nav>
          <button
            onClick={handleLogout}
            className="w-full flex items-center p-3 rounded-md hover:bg-blue-700 mt-auto mb-4"
          >
            <span className="mr-3">🚪</span>
            {!collapsed && <span>Logout</span>}
          </button>
          <button
            onClick={() => setCollapsed(!collapsed)}
            className="absolute -right-3 top-1/2 bg-blue-800 text-white w-6 h-12 rounded-r-md flex items-
center justify-center shadow-md"
```

```
          >
            {collapsed ? '»' : '«'}
          </button>
        </div>

        {/* Main content */}
        <div className="flex-1 flex flex-col overflow-hidden">
          <header className="bg-white shadow-sm h-16 flex items-center px-6">
            {/* Header content */}
          </header>
          <main className="flex-1 overflow-y-auto p-6">
            {renderContent()}
          </main>
        </div>
      </div>
    );
  };

  export default StateUnionDashboard;

  // FILE: frontend/src/config/api.js
  // API Configuration
  // Auto-detect production vs localhost
  const getApiBaseUrl = () => {
    if (import.meta.env.VITE_API_BASE_URL) {
      return import.meta.env.VITE_API_BASE_URL;
    }
    // Check if we're running on localhost
    if (typeof window !== 'undefined' && window.location.hostname === 'localhost') {
      return 'http://localhost:3001';
    }
    // Default to production URL
    return 'https://itu-f4bn.onrender.com';
  };

  const API_BASE_URL = getApiBaseUrl();

  export const API_ENDPOINTS = {
    // Admin endpoints
    ADMIN_LOGIN: `${API_BASE_URL}/api/admin/login`,
    GET_CONTACT: `${API_BASE_URL}/api/admin/getContact`,
    DELETE_CONTACT: (id) => `${API_BASE_URL}/api/admin/deleteContact/${id}`,
    GET_GALLERY: `${API_BASE_URL}/api/admin/getGallery`,
    UPLOAD_GALLERY: `${API_BASE_URL}/api/admin/uploadGallery`,
    DELETE_GALLERY: (id) => `${API_BASE_URL}/api/admin/deleteGallery/${id}`,
    GET_SLIDER: `${API_BASE_URL}/api/admin/getSlider`,
    UPLOAD_SLIDER: `${API_BASE_URL}/api/admin/uploadSlider`,
    DELETE_SLIDER: (id) => `${API_BASE_URL}/api/admin/deleteSlider/${id}`,
    GET_SELF_DEFENCE_SLIDER: `${API_BASE_URL}/api/admin/getSelfDefenceSlider`,
    UPLOAD_SELF_DEFENCE_SLIDER: `${API_BASE_URL}/api/admin/uploadSelfDefenceSlider`,
    DELETE_SELF_DEFENCE_SLIDER: (id) => `${API_BASE_URL}/api/admin/deleteSelfDefenceSlider/${id}`,
    GET_CATEGORY_SLIDER: `${API_BASE_URL}/api/admin/getCategorySlider`,
    UPLOAD_CATEGORY_SLIDER: `${API_BASE_URL}/api/admin/uploadCategorySlider`,
    UPDATE_CATEGORY_SLIDER: (id) => `${API_BASE_URL}/api/admin/editCategorySlider/${id}`,
    DELETE_CATEGORY_SLIDER: (id) => `${API_BASE_URL}/api/admin/deleteCategorySlider/${id}`,
    GET_ALL_NEWS: `${API_BASE_URL}/api/admin/getallNews`,
    GET_NEWS_BY_ID: (id) => `${API_BASE_URL}/api/admin/getNews/${id}`,
    ADD_NEWS: `${API_BASE_URL}/api/admin/addNews`,
    EDIT_NEWS: (id) => `${API_BASE_URL}/api/admin/editNews/${id}`,
    DELETE_NEWS: (id) => `${API_BASE_URL}/api/admin/deleteNews/${id}`,
    GET_FORM: `${API_BASE_URL}/api/admin/getForm`,
    APPROVE_FORM: `${API_BASE_URL}/api/admin/approveForm`,
    REJECT_FORM: `${API_BASE_URL}/api/admin/rejectForm`,
    DELETE_USER: `${API_BASE_URL}/api/admin/deleteUser`,
    SET_DISTRICT_HEAD: `${API_BASE_URL}/api/admin/set-district-head`,
    SET_STATE_HEAD: `${API_BASE_URL}/api/admin/set-state-head`,
    REMOVE_DISTRICT_HEAD: `${API_BASE_URL}/api/admin/remove-district-head`,
    REMOVE_STATE_HEAD: `${API_BASE_URL}/api/admin/remove-state-head`,
    GET_ORGANIZATIONS_BY_STATE: (stateName) =>
  `${API_BASE_URL}/api/admin/organizations/${encodeURIComponent(stateName)}`,

    // User endpoints
    CONTACT_US: `${API_BASE_URL}/api/user/contactus`,
    SUBMIT_FORM: (endpoint) => `${API_BASE_URL}/api/user/${endpoint}`,
    STATE_UNION_LOGIN: `${API_BASE_URL}/api/user/stateunionlogin`,
```

```
  // Player endpoints
  PLAYER_LOGIN: `${API_BASE_URL}/api/user/player/login`,
  REGISTER_PLAYERS: `${API_BASE_URL}/api/user/playerregistration`,
  GET_PLAYER_PROFILE: (id) => `${API_BASE_URL}/api/user/players/${id}`,
  UPDATE_PLAYER_PROFILE: (id) => `${API_BASE_URL}/api/user/players/${id}`,
  UPLOAD_PLAYER_PHOTO: (id) => `${API_BASE_URL}/api/user/players/${id}/photo`,
  GET_PLAYERS_BY_UNION: (unionId) => `${API_BASE_URL}/api/user/unions/${unionId}/players`,

  // Admin player endpoints
  GET_PLAYERS: `${API_BASE_URL}/api/admin/getPlayers`,
  APPROVE_PLAYERS: `${API_BASE_URL}/api/admin/approvePlayers`,
  REJECT_PLAYERS: `${API_BASE_URL}/api/admin/rejectPlayers`,
  DELETE_PLAYERS: `${API_BASE_URL}/api/admin/deletePlayers`,

  // Blog endpoints
  BLOG_POSTS: `${API_BASE_URL}/api/admin/blog/posts`,
  BLOG_FEATURED: `${API_BASE_URL}/api/admin/blog/featured`,
  BLOG_CATEGORIES: `${API_BASE_URL}/api/admin/blog/categories`,
  BLOG_TAGS: `${API_BASE_URL}/api/admin/blog/tags`,
  BLOG_POST: (slug) => `${API_BASE_URL}/api/admin/blog/post/${slug}`,
  BLOG_RELATED: (slug) => `${API_BASE_URL}/api/admin/blog/related/${slug}`,

  // Analytics endpoints
  DASHBOARD_ANALYTICS: `${API_BASE_URL}/api/admin/analytics/dashboard`,
  ENGAGEMENT_ANALYTICS: `${API_BASE_URL}/api/admin/analytics/engagement`,
  CONTENT_ANALYTICS: `${API_BASE_URL}/api/admin/analytics/content`,

  // Uploads
  UPLOADS: `${API_BASE_URL}/uploads`,

  // States and Districts endpoints
  GET_ALL_STATES: `${API_BASE_URL}/api/states`,
  GET_STATES_ONLY: `${API_BASE_URL}/api/states/states-only`,
  GET_UNION_TERRITORIES: `${API_BASE_URL}/api/states/union-territories`,
  GET_STATE: (stateName) => `${API_BASE_URL}/api/states/${encodeURIComponent(stateName)}`,
  GET_DISTRICTS: (stateName) =>
`${API_BASE_URL}/api/states/${encodeURIComponent(stateName)}/districts`,
  CREATE_STATE: `${API_BASE_URL}/api/states`,
  UPDATE_STATE_STATUS: (stateName) =>
`${API_BASE_URL}/api/states/${encodeURIComponent(stateName)}/status`,
  UPDATE_DISTRICTS: (stateName) =>
`${API_BASE_URL}/api/states/${encodeURIComponent(stateName)}/districts`,
  DELETE_STATE: (stateName) => `${API_BASE_URL}/api/states/${encodeURIComponent(stateName)}`,

  // Organization endpoints
  GET_ORGANIZATIONS_BY_DISTRICT: (stateName, districtName) =>
`${API_BASE_URL}/api/user/organizations/${encodeURIComponent(stateName)}/${encodeURIComponent(districtN
ame)}`,
  GET_STATE_UNION: (id) => `${API_BASE_URL}/api/user/stateunions/${id}`,
  UPDATE_STATE_UNION: (id) => `${API_BASE_URL}/api/user/stateunions/${id}`,
  UPLOAD_GENERAL_SECRETARY_IMAGE: (id) => `${API_BASE_URL}/api/user/stateunions/${id}/general-
secretary-image`,

  // Police Training endpoints
  GET_POLICE_TRAININGS: `${API_BASE_URL}/api/admin/police-training`,
  CREATE_POLICE_TRAINING: `${API_BASE_URL}/api/admin/police-training`,
  GET_POLICE_TRAINING_BY_ID: (id) => `${API_BASE_URL}/api/admin/police-training/${id}`,
  UPDATE_POLICE_TRAINING: (id) => `${API_BASE_URL}/api/admin/police-training/${id}`,
  DELETE_POLICE_TRAINING: (id) => `${API_BASE_URL}/api/admin/police-training/${id}`,
  DELETE_POLICE_TRAINING_PHOTO: (id, photoId) => `${API_BASE_URL}/api/admin/police-
training/${id}/photos/${photoId}`,
};

// Helper function for upload URLs
export const GET_UPLOAD_URL = (filename) => `${API_BASE_URL}/uploads/${filename}`;

export default API_BASE_URL;


// FILE: frontend/src/data/indianStatesDistricts.js
// Complete list of Indian States, Union Territories, and their Districts
// Data sourced from https://igod.gov.in/sg/district/states

export const indianStatesAndDistricts = {
```

```
    // States
    'Andhra Pradesh': {
      districts: [
        'Anantapur', 'Chittoor', 'East Godavari', 'Guntur', 'Krishna', 'Kurnool',
        'Nellore', 'Prakasam', 'Srikakulam', 'Visakhapatnam', 'Vizianagaram', 'West Godavari',
        'YSR Kadapa', 'Annamayya', 'Alluri Sitharama Raju', 'Bapatla', 'Eluru', 'Kakinada',
        'Konaseema', 'Nandyal', 'Palnadu', 'Parvathipuram Manyam', 'Sri Sathya Sai', 'Tirupati'
      ],
      type: 'State',
      active: false
    },
    'Arunachal Pradesh': {
      districts: [
        'Anjaw', 'Changlang', 'Dibang Valley', 'East Kameng', 'East Siang', 'Kamle',
        'Kra Daadi', 'Kurung Kumey', 'Lepa Rada', 'Lohit', 'Longding', 'Lower Dibang Valley',
        'Lower Siang', 'Lower Subansiri', 'Namsai', 'Pakke Kessang', 'Papum Pare', 'Shi Yomi',
        'Siang', 'Tawang', 'Tirap', 'Upper Siang', 'Upper Subansiri', 'West Kameng', 'West Siang'
      ],
      type: 'State',
      active: false
    },
    'Assam': {
      districts: [
        'Baksa', 'Barpeta', 'Biswanath', 'Bongaigaon', 'Cachar', 'Charaideo', 'Chirang',
        'Darrang', 'Dhemaji', 'Dhubri', 'Dibrugarh', 'Dima Hasao', 'Goalpara', 'Golaghat',
        'Hailakandi', 'Hojai', 'Jorhat', 'Kamrup', 'Kamrup Metropolitan', 'Karbi Anglong',
        'Karimganj', 'Kokrajhar', 'Lakhimpur', 'Majuli', 'Morigaon', 'Nagaon', 'Nalbari',
        'Sivasagar', 'Sonitpur', 'South Salmara-Mankachar', 'Tinsukia', 'Udalguri', 'West Karbi Anglong'
      ],
      type: 'State',
      active: false
    },
    'Bihar': {
      districts: [
        'Araria', 'Arwal', 'Aurangabad', 'Banka', 'Begusarai', 'Bhagalpur', 'Bhojpur',
        'Buxar', 'Darbhanga', 'East Champaran', 'Gaya', 'Gopalganj', 'Jamui', 'Jehanabad',
        'Kaimur', 'Katihar', 'Khagaria', 'Kishanganj', 'Lakhisarai', 'Madhepura', 'Madhubani',
        'Munger', 'Muzaffarpur', 'Nalanda', 'Nawada', 'Patna', 'Purnia', 'Rohtas', 'Saharsa',
        'Samastipur', 'Saran', 'Sheikhpura', 'Sheohar', 'Sitamarhi', 'Siwan', 'Supaul',
        'Vaishali', 'West Champaran'
      ],
      type: 'State',
      active: false
    },
    'Chhattisgarh': {
      districts: [
        'Balod', 'Baloda Bazar', 'Balrampur', 'Bastar', 'Bemetara', 'Bijapur', 'Bilaspur',
        'Dantewada', 'Dhamtari', 'Durg', 'Gariaband', 'Gaurela-Pendra-Marwahi', 'Janjgir-Champa',
        'Jashpur', 'Kabirdham', 'Kanker', 'Kondagaon', 'Korba', 'Koriya', 'Mahasamund',
        'Mungeli', 'Narayanpur', 'Raigarh', 'Raipur', 'Rajnandgaon', 'Sukma', 'Surajpur', 'Surguja'
      ],
      type: 'State',
      active: false
    },
    'Goa': {
      districts: ['North Goa', 'South Goa'],
      type: 'State',
      active: false
    },
    'Gujarat': {
      districts: [
        'Ahmedabad', 'Amreli', 'Anand', 'Aravalli', 'Banaskantha', 'Bharuch', 'Bhavnagar',
        'Botad', 'Chhota Udaipur', 'Dahod', 'Dang', 'Devbhoomi Dwarka', 'Gandhinagar',
        'Gir Somnath', 'Jamnagar', 'Junagadh', 'Kachchh', 'Kheda', 'Mahisagar', 'Mehsana',
        'Morbi', 'Narmada', 'Navsari', 'Panchmahal', 'Patan', 'Porbandar', 'Rajkot',
        'Sabarkantha', 'Surat', 'Surendranagar', 'Tapi', 'Vadodara', 'Valsad'
      ],
      type: 'State',
      active: true
    },
    'Haryana': {
      districts: [
        'Ambala', 'Bhiwani', 'Charkhi Dadri', 'Faridabad', 'Fatehabad', 'Gurugram',
        'Hisar', 'Jhajjar', 'Jind', 'Kaithal', 'Karnal', 'Kurukshetra', 'Mahendragarh',
        'Mewat', 'Palwal', 'Panchkula', 'Panipat', 'Rewari', 'Rohtak', 'Sirsa', 'Sonipat', 'Yamunanagar'
```

```
      ],
      type: 'State',
      active: false
    },
    'Himachal Pradesh': {
      districts: [
        'Bilaspur', 'Chamba', 'Hamirpur', 'Kangra', 'Kinnaur', 'Kullu', 'Lahaul and Spiti',
        'Mandi', 'Shimla', 'Sirmaur', 'Solan', 'Una'
      ],
      type: 'State',
      active: false
    },
    'Jharkhand': {
      districts: [
        'Bokaro', 'Chatra', 'Deoghar', 'Dhanbad', 'Dumka', 'East Singhbhum', 'Garhwa',
        'Giridih', 'Godda', 'Gumla', 'Hazaribagh', 'Jamtara', 'Khunti', 'Koderma', 'Latehar',
        'Lohardaga', 'Pakur', 'Palamu', 'Ramgarh', 'Ranchi', 'Sahibganj', 'Seraikela-Kharsawan',
        'Simdega', 'West Singhbhum'
      ],
      type: 'State',
      active: false
    },
    'Karnataka': {
      districts: [
        'Bagalkot', 'Ballari', 'Belagavi', 'Bengaluru Rural', 'Bengaluru Urban', 'Bidar',
        'Chamarajanagar', 'Chikballapur', 'Chikkamagaluru', 'Chitradurga', 'Dakshina Kannada',
        'Davangere', 'Dharwad', 'Gadag', 'Hassan', 'Haveri', 'Kalaburagi', 'Kodagu',
        'Kolar', 'Koppal', 'Mandya', 'Mysuru', 'Raichur', 'Ramanagara', 'Shivamogga',
        'Tumakuru', 'Udupi', 'Uttara Kannada', 'Vijayapura', 'Yadgir'
      ],
      type: 'State',
      active: true
    },
    'Kerala': {
      districts: [
        'Alappuzha', 'Ernakulam', 'Idukki', 'Kannur', 'Kasaragod', 'Kollam', 'Kottayam',
        'Kozhikode', 'Malappuram', 'Palakkad', 'Pathanamthitta', 'Thiruvananthapuram',
        'Thrissur', 'Wayanad'
      ],
      type: 'State',
      active: true
    },
    'Madhya Pradesh': {
      districts: [
        'Agar Malwa', 'Alirajpur', 'Anuppur', 'Ashoknagar', 'Balaghat', 'Barwani',
        'Betul', 'Bhind', 'Bhopal', 'Burhanpur', 'Chhatarpur', 'Chhindwara', 'Damoh',
        'Datia', 'Dewas', 'Dhar', 'Dindori', 'Guna', 'Gwalior', 'Harda', 'Hoshangabad',
        'Indore', 'Jabalpur', 'Jhabua', 'Katni', 'Khandwa', 'Khargone', 'Mandla', 'Mandsaur',
        'Morena', 'Narsinghpur', 'Neemuch', 'Niwari', 'Panna', 'Raisen', 'Rajgarh', 'Ratlam',
        'Rewa', 'Sagar', 'Satna', 'Sehore', 'Seoni', 'Shahdol', 'Shajapur', 'Sheopur',
        'Shivpuri', 'Sidhi', 'Singrauli', 'Tikamgarh', 'Ujjain', 'Umaria', 'Vidisha'
      ],
      type: 'State',
      active: false
    },
    'Maharashtra': {
      districts: [
        'Ahmednagar', 'Akola', 'Amravati', 'Aurangabad', 'Beed', 'Bhandara', 'Buldhana',
        'Chandrapur', 'Dhule', 'Gadchiroli', 'Gondia', 'Hingoli', 'Jalgaon', 'Jalna',
        'Kolhapur', 'Latur', 'Mumbai City', 'Mumbai Suburban', 'Nagpur', 'Nanded',
        'Nandurbar', 'Nashik', 'Osmanabad', 'Palghar', 'Parbhani', 'Pune', 'Raigad',
        'Ratnagiri', 'Sangli', 'Satara', 'Sindhudurg', 'Solapur', 'Thane', 'Wardha',
        'Washim', 'Yavatmal'
      ],
      type: 'State',
      active: true
    },
    'Manipur': {
      districts: [
        'Bishnupur', 'Chandel', 'Churachandpur', 'Imphal East', 'Imphal West', 'Jiribam',
        'Kakching', 'Kamjong', 'Kangpokpi', 'Noney', 'Pherzawl', 'Senapati', 'Tamenglong',
        'Tengnoupal', 'Thoubal', 'Ukhrul'
      ],
      type: 'State',
      active: false
```

```
    },
    'Meghalaya': {
      districts: [
        'East Garo Hills', 'East Jaintia Hills', 'East Khasi Hills', 'North Garo Hills',
        'Ri Bhoi', 'South Garo Hills', 'South West Garo Hills', 'South West Khasi Hills',
        'West Garo Hills', 'West Jaintia Hills', 'West Khasi Hills'
      ],
      type: 'State',
      active: false
    },
    'Mizoram': {
      districts: [
        'Aizawl', 'Champhai', 'Hnahthial', 'Khawzawl', 'Kolasib', 'Lawngtlai', 'Lunglei',
        'Mamit', 'Saiha', 'Saitual', 'Serchhip'
      ],
      type: 'State',
      active: false
    },
    'Nagaland': {
      districts: [
        'Chümoukedima', 'Dimapur', 'Kiphire', 'Kohima', 'Longleng', 'Mokokchung', 'Mon',
        'Niuland', 'Noklak', 'Peren', 'Phek', 'Shamator', 'Tuensang', 'Wokha', 'Zünheboto'
      ],
      type: 'State',
      active: false
    },
    'Odisha': {
      districts: [
        'Angul', 'Balangir', 'Baleshwar', 'Bargarh', 'Bhadrak', 'Boudh', 'Cuttack',
        'Deogarh', 'Dhenkanal', 'Gajapati', 'Ganjam', 'Jagatsinghapur', 'Jajpur',
        'Jharsuguda', 'Kalahandi', 'Kandhamal', 'Kendrapara', 'Kendujhar', 'Khordha',
        'Koraput', 'Malkangiri', 'Mayurbhanj', 'Nabarangpur', 'Nayagarh', 'Nuapada',
        'Puri', 'Rayagada', 'Sambalpur', 'Subarnapur', 'Sundargarh'
      ],
      type: 'State',
      active: false
    },
    'Punjab': {
      districts: [
        'Amritsar', 'Barnala', 'Bathinda', 'Faridkot', 'Fatehgarh Sahib', 'Fazilka',
        'Ferozepur', 'Gurdaspur', 'Hoshiarpur', 'Jalandhar', 'Kapurthala', 'Ludhiana',
        'Malerkotla', 'Mansa', 'Moga', 'Mohali', 'Muktsar', 'Pathankot', 'Patiala',
        'Rupnagar', 'Sangrur', 'Shaheed Bhagat Singh Nagar', 'Sri Muktsar Sahib', 'Tarn Taran'
      ],
      type: 'State',
      active: false
    },
    'Rajasthan': {
      districts: [
        'Ajmer', 'Alwar', 'Banswara', 'Baran', 'Barmer', 'Bharatpur', 'Bhilwara',
        'Bikaner', 'Bundi', 'Chittorgarh', 'Churu', 'Dausa', 'Dholpur', 'Dungarpur',
        'Hanumangarh', 'Jaipur', 'Jaisalmer', 'Jalore', 'Jhalawar', 'Jhunjhunu', 'Jodhpur',
        'Karauli', 'Kota', 'Nagaur', 'Pali', 'Pratapgarh', 'Rajsamand', 'Sawai Madhopur',
        'Sikar', 'Sirohi', 'Sri Ganganagar', 'Tonk', 'Udaipur'
      ],
      type: 'State',
      active: false
    },
    'Sikkim': {
      districts: ['East Sikkim', 'North Sikkim', 'South Sikkim', 'West Sikkim'],
      type: 'State',
      active: false
    },
    'Tamil Nadu': {
      districts: [
        'Ariyalur', 'Chengalpattu', 'Chennai', 'Coimbatore', 'Cuddalore', 'Dharmapuri',
        'Dindigul', 'Erode', 'Kallakurichi', 'Kanchipuram', 'Kanyakumari', 'Karur',
        'Krishnagiri', 'Madurai', 'Mayiladuthurai', 'Nagapattinam', 'Namakkal', 'Nilgiris',
        'Perambalur', 'Pudukkottai', 'Ramanathapuram', 'Ranipet', 'Salem', 'Sivaganga',
        'Tenkasi', 'Thanjavur', 'Theni', 'Thoothukudi', 'Tiruchirappalli', 'Tirunelveli',
        'Tirupathur', 'Tiruppur', 'Tiruvallur', 'Tiruvannamalai', 'Vellore', 'Viluppuram',
        'Virudhunagar'
      ],
      type: 'State',
      active: true
```

```
    },
    'Telangana': {
      districts: [
        'Adilabad', 'Bhadradri Kothagudem', 'Hyderabad', 'Jagtial', 'Jangaon', 'Jayashankar Bhupalpally',
        'Jogulamba Gadwal', 'Kamareddy', 'Karimnagar', 'Khammam', 'Komaram Bheem Asifabad',
        'Mahabubabad', 'Mahabubnagar', 'Mancherial', 'Medak', 'Medchal-Malkajgiri', 'Mulugu',
        'Nagarkurnool', 'Nalgonda', 'Narayanpet', 'Nirmal', 'Nizamabad', 'Peddapalli',
        'Rajanna Sircilla', 'Rangareddy', 'Sangareddy', 'Siddipet', 'Suryapet', 'Vikarabad',
        'Wanaparthy', 'Warangal Rural', 'Warangal Urban', 'Yadadri Bhuvanagiri'
      ],
      type: 'State',
      active: false
    },
    'Tripura': {
      districts: [
        'Dhalai', 'Gomati', 'Khowai', 'North Tripura', 'Sepahijala', 'South Tripura',
        'Unakoti', 'West Tripura'
      ],
      type: 'State',
      active: false
    },
    'Uttar Pradesh': {
      districts: [
        'Agra', 'Aligarh', 'Allahabad', 'Ambedkar Nagar', 'Amethi', 'Amroha', 'Auraiya',
        'Azamgarh', 'Baghpat', 'Bahraich', 'Ballia', 'Balrampur', 'Banda', 'Barabanki',
        'Bareilly', 'Basti', 'Bhadohi', 'Bijnor', 'Budaun', 'Bulandshahr', 'Chandauli',
        'Chitrakoot', 'Deoria', 'Etah', 'Etawah', 'Faizabad', 'Farrukhabad', 'Fatehpur',
        'Firozabad', 'Gautam Buddha Nagar', 'Ghaziabad', 'Ghazipur', 'Gonda', 'Gorakhpur',
        'Hamirpur', 'Hapur', 'Hardoi', 'Hathras', 'Jalaun', 'Jaunpur', 'Jhansi', 'Kannauj',
        'Kanpur Dehat', 'Kanpur Nagar', 'Kasganj', 'Kaushambi', 'Kheri', 'Kushinagar',
        'Lakhimpur Kheri', 'Lalitpur', 'Lucknow', 'Maharajganj', 'Mahoba', 'Mainpuri',
        'Mathura', 'Mau', 'Meerut', 'Mirzapur', 'Moradabad', 'Muzaffarnagar', 'Pilibhit',
        'Pratapgarh', 'Prayagraj', 'Raebareli', 'Rampur', 'Saharanpur', 'Sambhal', 'Sant Kabir Nagar',
        'Shahjahanpur', 'Shamli', 'Shravasti', 'Siddharthnagar', 'Sitapur', 'Sonbhadra',
        'Sultanpur', 'Unnao', 'Varanasi'
      ],
      type: 'State',
      active: false
    },
    'Uttarakhand': {
      districts: [
        'Almora', 'Bageshwar', 'Chamoli', 'Champawat', 'Dehradun', 'Haridwar', 'Nainital',
        'Pauri Garhwal', 'Pithoragarh', 'Rudraprayag', 'Tehri Garhwal', 'Udham Singh Nagar', 'Uttarkashi'
      ],
      type: 'State',
      active: false
    },
    'West Bengal': {
      districts: [
        'Alipurduar', 'Bankura', 'Birbhum', 'Cooch Behar', 'Dakshin Dinajpur', 'Darjeeling',
        'Hooghly', 'Howrah', 'Jalpaiguri', 'Jhargram', 'Kalimpong', 'Kolkata', 'Malda',
        'Murshidabad', 'Nadia', 'North 24 Parganas', 'Paschim Bardhaman', 'Paschim Medinipur',
        'Purba Bardhaman', 'Purba Medinipur', 'Purulia', 'South 24 Parganas', 'Uttar Dinajpur'
      ],
      type: 'State',
      active: false
    },

    // Union Territories
    'Andaman and Nicobar Islands': {
      districts: ['Nicobar', 'North and Middle Andaman', 'South Andaman'],
      type: 'Union Territory',
      active: false
    },
    'Chandigarh': {
      districts: ['Chandigarh'],
      type: 'Union Territory',
      active: false
    },
    'Dadra and Nagar Haveli and Daman and Diu': {
      districts: ['Dadra and Nagar Haveli', 'Daman', 'Diu'],
      type: 'Union Territory',
      active: false
    },
    'Delhi': {
```

```
    districts: [
      'Central Delhi', 'East Delhi', 'New Delhi', 'North Delhi', 'North East Delhi',
      'North West Delhi', 'Shahdara', 'South Delhi', 'South East Delhi', 'South West Delhi', 'West
Delhi'
    ],
    type: 'Union Territory',
    active: true
  },
  'Jammu and Kashmir': {
    districts: [
      'Anantnag', 'Bandipora', 'Baramulla', 'Budgam', 'Doda', 'Ganderbal', 'Jammu',
      'Kathua', 'Kishtwar', 'Kulgam', 'Kupwara', 'Poonch', 'Pulwama', 'Rajouri',
      'Ramban', 'Reasi', 'Samba', 'Shopian', 'Srinagar', 'Udhampur'
    ],
    type: 'Union Territory',
    active: false
  },
  'Ladakh': {
    districts: ['Kargil', 'Leh'],
    type: 'Union Territory',
    active: false
  },
  'Lakshadweep': {
    districts: ['Lakshadweep'],
    type: 'Union Territory',
    active: false
  },
  'Puducherry': {
    districts: ['Karaikal', 'Mahe', 'Puducherry', 'Yanam'],
    type: 'Union Territory',
    active: false
  }
};

// Helper function to get all states and UTs
export const getAllStatesAndUTs = () => {
  return Object.keys(indianStatesAndDistricts);
};

// Helper function to get districts for a state/UT
export const getDistrictsForState = (stateName) => {
  return indianStatesAndDistricts[stateName]?.districts || [];
};

// Helper function to get state/UT info
export const getStateInfo = (stateName) => {
  return indianStatesAndDistricts[stateName] || null;
};

// Get all states (excluding UTs)
export const getAllStates = () => {
  return Object.entries(indianStatesAndDistricts)
    .filter(([_, info]) => info.type === 'State')
    .map(([name, _]) => name);
};

// Get all Union Territories
export const getAllUnionTerritories = () => {
  return Object.entries(indianStatesAndDistricts)
    .filter(([_, info]) => info.type === 'Union Territory')
    .map(([name, _]) => name);
};



// FILE: frontend/src/main.jsx
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import { BrowserRouter } from "react-router-dom";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <BrowserRouter>
    <App />
```

```
      </BrowserRouter>
    </React.StrictMode>
  );


// FILE: frontend/src/utils/auth.js
const TOKEN_KEY = 'accessToken';
const REFRESH_TOKEN_KEY = 'refreshToken';
const USER_DATA_KEY = 'userData';

/**
 * Store tokens and user data
 */
export const setAuthData = (accessToken, refreshToken, userData) => {
  localStorage.setItem(TOKEN_KEY, accessToken);
  if (refreshToken) {
    localStorage.setItem(REFRESH_TOKEN_KEY, refreshToken);
  }
  if (userData) {
    localStorage.setItem(USER_DATA_KEY, JSON.stringify(userData));
  }
};

/**
 * Get access token
 */
export const getAccessToken = () => {
  return localStorage.getItem(TOKEN_KEY);
};

/**
 * Get refresh token
 */
export const getRefreshToken = () => {
  return localStorage.getItem(REFRESH_TOKEN_KEY);
};

/**
 * Get user data
 */
export const getUserData = () => {
  const data = localStorage.getItem(USER_DATA_KEY);
  return data ? JSON.parse(data) : null;
};

/**
 * Clear all auth data
 */
export const clearAuthData = () => {
  localStorage.removeItem(TOKEN_KEY);
  localStorage.removeItem(REFRESH_TOKEN_KEY);
  localStorage.removeItem(USER_DATA_KEY);
  // Also clear legacy keys
  localStorage.removeItem('playerData');
  localStorage.removeItem('stateUnionId');
};

/**
 * Check if user is authenticated
 */
export const isAuthenticated = () => {
  return !!getAccessToken();
};

/**
 * Get authorization header for API requests
 */
export const getAuthHeader = () => {
  const token = getAccessToken();
  return token ? { Authorization: `Bearer ${token}` } : {};
};

/**
 * Refresh access token
 */
```

```
export const refreshAccessToken = async () => {
  const refreshToken = getRefreshToken();
  if (!refreshToken) {
    throw new Error('No refresh token available');
  }

  const API_BASE_URL = import.meta.env.VITE_API_BASE_URL ||
    (window.location.hostname === 'localhost' ? 'http://localhost:3001' : 'https://itu-
f4bn.onrender.com');

  try {
    const response = await fetch(`${API_BASE_URL}/api/user/refresh-token`, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({ refreshToken }),
    });

    if (!response.ok) {
      throw new Error('Failed to refresh token');
    }

    const data = await response.json();
    if (data.success && data.accessToken) {
      localStorage.setItem(TOKEN_KEY, data.accessToken);
      return data.accessToken;
    }

    throw new Error('Invalid refresh token response');
  } catch (error) {
    clearAuthData();
    throw error;
  }
};



// FILE: frontend/src/utils/axiosConfig.js
import axios from 'axios';
import { getAccessToken, refreshAccessToken, clearAuthData } from './auth';

const API_BASE_URL = import.meta.env.VITE_API_BASE_URL ||
  (window.location.hostname === 'localhost' ? 'http://localhost:3001' : 'https://itu-
f4bn.onrender.com');

// Create axios instance
const api = axios.create({
  baseURL: API_BASE_URL,
  withCredentials: true,
  headers: {
    'Content-Type': 'application/json',
  },
});

// Request interceptor - Add token to requests
api.interceptors.request.use(
  (config) => {
    const token = getAccessToken();
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  (error) => {
    return Promise.reject(error);
  }
);

// Response interceptor - Handle token refresh
api.interceptors.response.use(
  (response) => {
    return response;
  },
  async (error) => {
```

```
    const originalRequest = error.config;

    // If error is 401 and we haven't tried to refresh yet
    if (error.response?.status === 401 && !originalRequest._retry) {
      originalRequest._retry = true;

      try {
        // Try to refresh the token
        const newAccessToken = await refreshAccessToken();

        // Retry the original request with new token
        originalRequest.headers.Authorization = `Bearer ${newAccessToken}`;
        return api(originalRequest);
      } catch (refreshError) {
        // Refresh failed, clear auth and redirect to login
        clearAuthData();
        if (window.location.pathname !== '/login') {
          window.location.href = '/login';
        }
        return Promise.reject(refreshError);
      }
    }

    return Promise.reject(error);
  }
);

export default api;



// FILE: frontend/vite.config.js
import { defineConfig } from 'vite'
import tailwindcss from '@tailwindcss/vite'
export default defineConfig({
  plugins: [
    tailwindcss(),
  ],
  build: {
    rollupOptions: {
      onwarn(warning, warn) {
        // Suppress "use client" directive warnings from framer-motion
        if (warning.code === 'MODULE_LEVEL_DIRECTIVE') {
          return;
        }
        warn(warning);
      },
    },
  },
  logLevel: 'warn', // Reduce verbose warnings
})
```